Prueba

Conceptos <u>erróneos</u> de "prueba":

- "Probar es demostrar que no hay errores presentes en un programa" → Esto es imposible.
- "Probar es el proceso que lleva a confiar en que un programa hará lo que se supone que debe hacer" → Esto es incompleto ya que se debe mostrar también que el programa *no* hace lo que *no* se supone que debe hacer.

Prueba: Proceso destructivo de tratar de encontrar errores en un programa. El caso exitoso de prueba es aquel que encuentra un error encubierto. Un buen caso de prueba es aquel que tiene una elevada probabilidad de encontrar un error aun no descubierto. Se establece finalmente un grado de confiabilidad en el programa alegando que el programa hace lo que se supone que debe hacer y no hace lo que no se supone que debe hacer, luego de una diligente exploración de errores.

Economía de las pruebas: El ser imposible y no práctico encontrar todos los errores en un programa tiene consecuencia sobre la economía de la prueba, sobre las suposiciones que el operador de la prueba deberá hacer sobre el programa y sobre la forma en que se diseñaron los casos de prueba.

Principios de prueba:

- Un caso de prueba debe constar de dos componentes:
  - Una descripción de los datos de entrada al programa.
  - Una descripción precisa de la salida esperada para esos datos de entrada.
- Un programador no debe probar su propio programa:
  - Falta de actitud mental para querer encontrar errores en sus programas.
  - Falta de actitud destructiva ante tanta actitud constructiva que tiene con su programa.
  - Falsa interpretación de un requerimiento. Si el programador no captó bien el requerimiento, en una prueba propia este error no se develará.
  - El debugging sí lo debe hacer el propio programador.
- Inspeccionar cuidadosamente el caso de prueba ya que el error se puede "camuflar".
- Los casos de prueba deben ser escritos tanto para condiciones de entradas inválidas e inesperadas como para condiciones válidas y esperadas.
- Examinar un programa para comprobar que no hace lo que se supone que debe hacer es sólo la mitad del problema. La otra mitad consiste en ver si el programa hace lo que no se supone que debe hacer.
- Evite los casos de prueba desechables: representan una inversión valiosa. Luego de una corrección, los casos de prueba deben ser reintentados aunque cueste trabajo. Si la modificación causa un error en una parte del programa que anteriormente era correcta, este error a menudo pasa desapercibido.
- No planear un esfuerzo de prueba con la suposición de que no se encontraran errores.
- Tener en cuenta que la probabilidad de encontrar errores adicionales en una sección del programa es proporcional al número de errores ya encontrados en esa misma sección.
- Las pruebas constituyen una tarea altamente creativa y son un desafío intelectual.

Aspectos estratégicos:

Una estrategia de software triunfará cuando quienes prueben el software:

- *Especifican los requerimientos del producto en forma cuantificable mucho antes de comenzar con las pruebas,* de modo que los resultados de las pruebas no sean ambiguos.
- *Establecen de manera explícita los objetivos de las pruebas.*
- Entienden a los usuarios del software y desarrollan un perfil para cada categoría del usuario.
- Desarrollan un plan de prueba que enfatice "prueba de ciclo rápido".
- Construyen software "robusto" que esté diseñado para probarse a sí mismo.
- Usan revisiones técnicas efectivas como filtro previo a las pruebas.
- Realizan revisiones técnicas para valorar la estrategia de prueba y los casos de prueba.
- Desarrollan un enfoque de mejora continuo para el proceso de prueba.

Pruebas por módulos:

Esta sección asume que el sistema ha sido diseñado según el método estructurado, por lo que necesariamente se cuenta con un conjunto de módulos pequeñamente manejables de alta cohesión (independencia funcional) y bajo acople (sin interdependencia).

La prueba por módulos (o prueba por unidades) consiste en comparar la función de un módulo con alguna especificación funcional o interfaz que lo define.

Ventajas:

- Permite manejar con mayor facilidad las posibilidades combinatorias de las pruebas, puesto que la atención es inicialmente enfocada a unidades menores de programa.
- Facilita la tarea de localización y corrección de errores (debugging), puesto que cuando se descubre un error, éste estará presente en un módulo en particular.
- Introduce un cierto paralelismo en el proceso de prueba puesto que da la oportunidad de probar simultáneamente múltiples módulos.

Recordar que el propósito no es mostrar que el módulo funciona correctamente sino que contiene errores.

Diseño de casos de prueba:

Cuando se diseñan casos de prueba para una prueba por módulos, debe disponerse de dos tipos de información:

a. El código fuente del módulo.

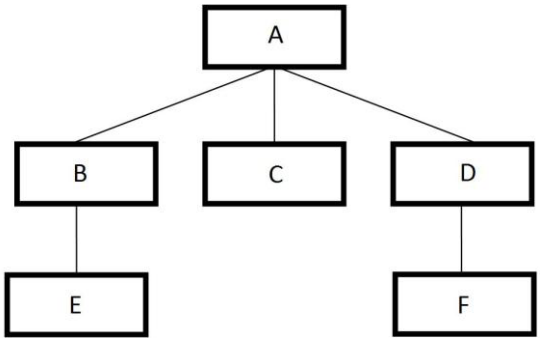
b. La especificación del módulo (parámetros de entrada y salida).

Procedimiento para prueba por módulos:

1. Se analiza la lógica del programa usando métodos de caja blanca (conociendo el código). Se aplican pruebas sobre el punto "a." anterior.
2. Se complementan usando métodos de caja negra a las especificaciones del módulo. Se aplican pruebas sobre "b."

Formas de prueba:

Sea el siguiente programa de 6 módulos A, B, C, D, E y F.



Esta distribución indica que el modulo A, para su ejecución, llama a B, C y D. B llama a E y D a F.
Para realizarle pruebas a un módulo se debe tener:

- Un conjunto preparado de casos de prueba.
- La manera en la cual se combinan los distintos módulos. Maneras de tratar este último ítem:

  - Pruebas de integración:
    Pruebas incrementales: Combinar el siguiente módulo a probar con el conjunto de módulos probados previamente antes de que se pruebe el sistema.
    - Descendente: Se comienza por la parte superior del programa (módulo A).
    - Ascendente: Se comienza por la parte inferior del programa (módulos E, C y F).
    - Ventajas:
      - Para el caso ascendente se codifica el modulo impulsor para cada módulo que se está probando pero no requiere de auxiliares. Para el caso descendente no requiere módulos impulsores. Esto es porque cada avance incluye al módulo ya probado.
      - Los errores en interfaces se van descubriendo a medida se van probando los módulos porque se van incluyendo los módulos ya probados. Entonces se sabe dónde está el error y el debugging es más fácil.
      - Con cada prueba, al usarse la combinación de los módulos ya probados, estos últimos están más "expuestos" porque se usan en las pruebas posteriores. Entonces la prueba resulta más profunda.
    - Desventajas:
      - Requiere más tiempo de máquina que la no incremental al ir agregando ejecuciones de módulos ya probados. Sin embargo esto resulta atenuado ya que hoy en día el hardware es potente.

    Pruebas no incrementales: Es muy sencilla y se recomienda para proyectos pequeños. Consiste en probar cada módulo por separado de manera similar a la integración incremental, pero después de terminar con los módulos independientes se continúa probándolos todos juntos como un todo.

    - Ventajas:
      - No se necesita ningún tipo de trabajo adicional, ni planificar al orden de integración, ni crear módulos impulsores, ni módulos ficticios subordinados.
    - Desventajas:
      - No se tiene noción de la comunicación hasta el final, en ningún momento se dispone de un producto para mostrar.
      - Los errores en interfaces se descubren recién al final de la prueba donde se combinan todos los módulos, y el debugging se hace difícil porque encontrar la causa de los errores puede resultar mucho más compleja.
      - El hecho de presentar las pruebas de una vez hace que las sesiones de prueba sean más largas y tediosas.

- Prueba de regresión

Cualquier tipo de pruebas de software que intentan descubrir errores causados por la realización de un cambio en el programa. En el contexto de una estrategia de prueba de integración, la prueba de regresión es la nueva ejecución de algún subconjunto de prueba que ya se realizó a fin de asegurar que los cambios, al agregar un nuevo módulo, no propagaron efectos colaterales no deseados. En un contexto más amplio, las pruebas exitosas (de cualquier tipo) dan como resultado el descubrimiento de errores, que al corregirse cambian algún aspecto de la configuración del software. Las pruebas de regresión ayudan a garantizar que los cambios no introducen comportamiento no planeado o errores adicionales.

- Prueba de humo

La prueba de humo es un enfoque de integración para proyectos críticos en el tiempo que permite al equipo del software valorar el proyecto de manera frecuente. En esencia, el enfoque de prueba de humo abarca las siguientes actividades:

- Los componentes de software traducidos en código se integran en una construcción. Una construcción incluye todos los archivos de datos, bibliotecas, módulos reutilizables y componentes sometidos a ingeniería que se requieren para implementar una o más funciones del producto.

- Se diseña una serie de pruebas para exponer los errores que evitaran a la construcción realizar adecuadamente su función. La intención debe ser descubrir errores “paralizantes” que tengan la mayor probabilidad de retrasar el proyecto.

- La construcción se integra con otras construcciones, y todo el producto (en su forma actual) se somete a prueba de humo diariamente. El enfoque de integración puede ser descendente o ascendente.

- **Prueba por módulos en el contexto OO**

En software orientado a objeto el concepto de unidad equivale a la clase de objeto debido a que operaciones pueden existir como parte de clases diferentes, donde el contexto (atributos y operaciones privados) cambia y hace necesario probar la operación en el contexto de cada clase. A diferencia de la prueba de unidad convencional ya no es posible probar una operación en aislamiento sino más bien como parte de una clase.

- **Prueba de integración en el contexto OO**

Puesto que el software orientado a objeto no tiene una estructura de control jerárquico obvia, las estrategias tradicionales descendentes y ascendentes tienen poco significado. Existen dos estrategias diferentes para la prueba de integración de los sistemas OO.

- *La prueba basada en hebra*, integra el conjunto de clases requeridas para responder a una entrada o evento para el sistema. Cada hebra se integra y prueba de manera individual. La prueba de regresión se aplica para asegurar que no ocurran efectos colaterales.
- *La prueba basada en uso*, comienza la construcción del sistema probando clases llamadas "independientes" que usan muy pocas clases servidor (si es que usan algunas). Después de probar las clases independientes, se prueba la siguiente capa de clases llamadas dependientes, que usan las clases independientes. Esta secuencia de probar capas de clases dependientes continua hasta que se construye todo el sistema.

La prueba de grupo es un paso en la prueba de integración del software OO. Aquí, un grupo de clases colaboradoras se ejercita al diseñar casos de prueba que intente descubrir errores en las colaboraciones.

Técnicas para llevar a cabo pruebas:

Se ha demostrado que la prueba humana (probar un programa leyéndolo sin usar la computadora) es muy efectiva para encontrar errores por varias razones:

- El programador se siente más presionado cuando se corrigen errores encontrados mediante pruebas efectuadas con computadora que cuando se trata de errores encontrados antes.
- Mientras antes se detecten los errores, menos cuesta corregirlos, y más probable es corregirlos bien.
- Las pruebas humanas y por computadora son complementarias. Se deben efectuar ambas.
  Los métodos que aquí se describen se aplicarán entre el momento en que el módulo es codificado y el comienzo de las pruebas basadas en el uso de computadoras.

- Inspecciones:

  Conjunto de procedimientos y técnicas para la lectura de programas efectuada por un equipo de personas. Se deben contar idealmente con cuatro personas:

  - Moderador: Autoridad sobre el resto, programador experto, no debe ser el autor del programa ni estar al tanto de los detalles.
    Su tarea consiste en distribuir el material necesario para las sesiones de inspección, coordinar los horarios, dirigir las sesiones, tomar nota de todo error encontrado y asegurarse de que los errores sean posteriormente corregidos.
  - Programador.
  - Diseñador del programa.
  - Especialista en pruebas.

  Procedimiento para la inspección:

  Con varios días de anticipación, el moderador distribuye el programa y las especificaciones al grupo. Durante las sesiones se efectuarán dos actividades:

  - Se pide al programador que describa, sentencia por sentencia, la lógica del programa. Se le formulan preguntas insistentes hasta determinar si existen errores. En esta etapa el programador encuentra más errores que el resto. Leer su propio código en voz alta funciona.
  - Se analiza el programa por medio de una ayuda memoria preparada previamente (que incluye errores que se sabe son frecuentes). El moderador se asegura que se siga el objetivo de encontrar errores y no de intentar corregirlos.

    El programador no debe considerarlo como un ataque personal porque se pone a la defensiva y no sirve. Los resultados deben ser confidenciales. Este proceso es beneficioso para todos los integrantes de la reunión ya que reciben retroalimentación entre todos.

  Al finalizar la sesión:

  - El programador recibe una lista con los errores encontrados:
    - Si son numerosos o requieren correcciones sustanciales, el moderador puede organizar una reinspección luego de la corrección de estos errores.
    - Esta lista es analizada, clasificada por categoría y empleada para afinar la lista ayuda-memoria empleada, y mejorar la eficacia de futuras inspecciones.

Diseño de los casos de prueba

En el diseño de los casos de prueba se debe plantear: *¿Cuál es el subconjunto de todos los posibles casos de prueba que tiene la mayor probabilidad de detectar el mayor número posible de*

errores?

Es posible desarrollar una prueba razonablemente rigurosa usando ciertas metodologías orientadas hacia la prueba de tipo caja negra y entonces complementar los casos de prueba así obtenidos examinando la lógica del programa (es decir, usando métodos de caja blanca).

Prueba de cobertura lógica (caja blanca):

Se ocupa del grado en que los casos de prueba ejercitan o cubren la lógica (código fuente) del programa.

- Cobertura de decisión o ramificación: Este criterio establece que es necesario escribir un número suficiente de casos de pruebas como para que cada decisión tenga, por lo menos una vez, un resultado verdadero y uno falso. Otra forma de expresar lo mismo es decir que cada dirección de una rama debe ser recorrida por lo menos una vez.
- Cobertura de condición: Es necesario un número suficiente de casos de prueba de modo tal que cada condición en una decisión tenga todos los resultados posibles, al menos una vez.
- Cobertura decisión/condición: Es necesario que cada condición en una determinada decisión tenga todas las posibles salidas y que cada punto de entrada sea activado al menos una vez.

Pruebas de caja negra

Particiones de equivalencia:

Consiste en seleccionar el subconjunto de posibles entradas que tenga la mayor probabilidad de encontrar gran parte de los errores.

Cada caso de prueba debe invocar el mayor número posible de condiciones de entrada diferentes a fin de minimizar el número total necesario de casos.

Debe tratarse de partir el dominio de entrada de un programa en un número finito de clases de equivalencia, de modo que pueda razonablemente suponerse que la prueba de un valor representativo de cada clase sea equivalente a la prueba de cualquier valor dentro de esa clase.

Cuenta de dos pasos:

- Identificación de las clases de equivalencia: Son identificadas tomando cada condición de entrada y dividiéndola en dos o más grupos.
- Identificación de los casos de prueba:
  - Se asigna un id a cada clase.
  - Se escribe un nuevo caso de prueba que cubra tantas clases válidas no cubiertas posibles hasta que todas las clases hayan sido cubiertas.
  - Hasta que todas las clases de equivalencia inválidas hayan sido cubiertas por casos de prueba, escribir un caso de prueba para cubrir una y sólo una, de las clases de equivalencia no cubiertas.

Análisis de valores límite

La experiencia muestra que los casos de prueba que exploran las condiciones límite producen mejor resultado que aquellos que no lo hacen. Las condiciones límites son las situaciones que se hayan directamente arriba y debajo de los márgenes de las clases de equivalencia de entrada y de salida. El análisis de valores límite difiere en dos aspectos de la partición de equivalencia.

- Más que seleccionar cualquier elemento como representativo de una clase de equivalencia, el análisis de valores límite requiere que se seleccionen uno o más elementos de tal manera que cada margen de la clase de equivalencia sea sujeto de una prueba.

- Más que concentrar la atención en las condiciones de entrada (espacio de entrada), los casos de prueba se derivan también considerando el espacio de resultados (es decir clases de equivalencia de salida).

Algunas reglas son:

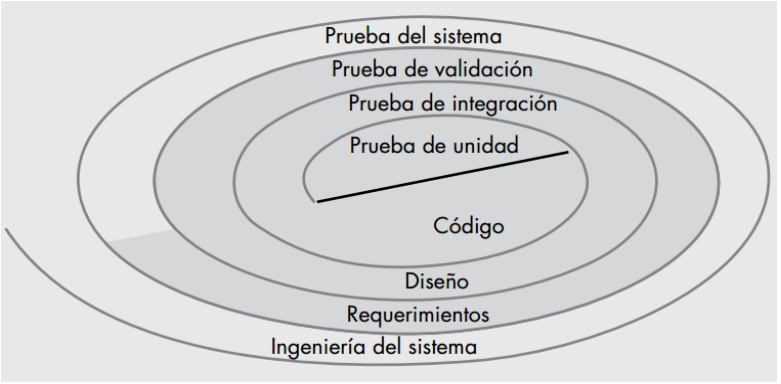
- Si una condición de entrada especifica un rango de valores, escribir casos de prueba para los extremos del rango, y casos de prueba inválidos para situaciones más allá de los extremos.
- Si una condición de entrada especifica un número de valores, escribir casos de prueba para los números de valores máximo y mínimo y uno por encima y por debajo de esos valores.
- Si la entrada o la salida de un programa es un conjunto ordenado, enfocar la atención en el primero y en el último de los elementos del conjunto.

Conjetura de errores

Capacidad de encontrar ciertos tipos de errores, a través de intuición y experiencia, sin usar una metodología particular. Se escribe una lista de posibles errores y se arman casos de prueba para ellos, o a partir de suposiciones posibles que el programador podría haber hecho al leer las especificaciones.

Estrategia de prueba del software. Visión general

La prueba de módulo o unidad comienza en el vértice de la espiral y se concentra en cada unidad del software como se implementó en el código fuente. La prueba avanza al moverse hacia afuera a lo largo de la espiral, hacia la prueba de integración donde el enfoque se centra en el diseño y la construcción de la arquitectura del software. Al dar otra vuelta hacia fuera de la Espiral, se encuentra la prueba de validación, donde los Requerimientos establecidos como parte de su modelado revalidan confrontándose con el software que se construyó. Finalmente, se llega a la prueba del sistema, donde el software y otros elementos del sistema se prueban como un todo.



Estrategias de prueba para software convencional

Una estrategia de prueba que eligen la mayoría de los equipos de software se coloca entre los dos extremos. Toma una visión incremental de las pruebas, comenzando con la de módulos de programa individuales, avanza hacia pruebas diseñadas para facilitar la integración de las unidades y culmina con pruebas que ejercitan el sistema construido.

Se deben combinar todas las metodología de diseño nombradas, porque cada una por sí sola no constituyen un conjunto perfecto de casos de prueba. *Generalmente un 90% resulta en pruebas de caja negra contra 10% de pruebas de caja blanca.* Estrategia razonable:

1. En todos los casos usar análisis de valores límite. Recordar que estos límites son aplicados a la entrada y a la salida.
2. Identificar las clases válidas e inválidas de equivalencia para la entrada y la salida, y suplementar, si es necesario, con los casos de prueba indicados más arriba.

3. Usar la técnica de conjetura de error para agregar casos de prueba adicionales.
4. Examinar la lógica del programa con vistas al conjunto de casos de prueba. Usar los criterios de cobertura de decisión, de condición, de decisión/condición, o de condición múltiple (el último es el más completo). Si el criterio de cobertura no ha sido llenado, añadir suficientes casos de prueba para lograr que el criterio sea satisfecho.

Estrategias de prueba para webapps

Los siguientes pasos resumen el enfoque:

- El módulo de contenido para webapp se revisa para describir errores.
- El módulo de interfaz se revisa para garantizar que todos los casos de uso pueden adecuarse.
- El módulo de diseño para la webapp se revisa para descubrir errores de navegación.
- La interfaz de usuario se prueba para descubrir errores en los mecanismos de presentación y/o navegación.
- A cada componente funcional se le aplica una prueba de unidad.
- Se prueba la navegación a lo largo de toda la arquitectura.
- La webapp se implementa en varias configuraciones ambientales diferentes y se prueba en su compatibilidad con cada configuración.
- Las pruebas de seguridad se realizan con la intención de explotar vulnerabilidades en la webapp o dentro de su ambiente.
- Se realizan pruebas de rendimiento.
- La webapp se prueba mediante una población de usuarios finales controlada y monitoreada. Los resultados de su interacción con el sistema se evalúan por errores de contenido y navegación, preocupaciones de facilidad de uso, preocupaciones de compatibilidad, así como confiabilidad y rendimiento de la webapp.

Pruebas de nivel superior

Comienzan después de que se completa la prueba de módulos, el software está completamente ensamblado como un paquete y los errores de interfaz se descubrieron y corrigieron. Esta es la prueba de nivel superior (probar el programa armado y no cada módulo). Tipos de prueba de nivel superior:

- Pruebas de validación (o de función): La prueba de función es el proceso de tratar de encontrar discrepancias entre el sistema y su especificación externa (descripción precisa del comportamiento del programa desde el punto de vista del usuario). Las pruebas se enfocan en las acciones visibles para el usuario y las salidas del sistema reconocibles por el usuario. La validación es exitosa cuando el software funciona en una forma que cumpla con las expectativas razonables del cliente.

- Prueba de aceptación: Proceso de comparar el programa con sus requerimientos iniciales y con las necesidades actuales del usuario final. La realiza el usuario final, tratando de mostrar que el programa no satisface el contrato.
    - Pruebas alfa y beta

Virtualmente es imposible que un desarrollador de software prevea como usará el cliente realmente un programa. Las instrucciones para usarlo pueden malinterpretarse; regularmente pueden usarse combinaciones extrañas de datos; la salida que parecía clara a quien realizo la

prueba puede ser ininteligible para un usuario. Cuando se construye software a la medida para un cliente, se realiza una serie de pruebas de aceptación a fin de permitir al cliente validar todos los requerimientos.

La prueba alfa se lleva a cabo en el sitio del desarrollador por un grupo representativo de usuarios finales. El software se usa en un escenario natural con el desarrollador "mirando sobre el hombro" de los usuarios y registrando los errores y problemas de uso. Las pruebas alfa se realizan en un ambiente controlado.

La prueba beta se realiza en uno o más sitios del usuario final. A diferencia de la prueba alfa, por lo general el desarrollador no está presente. Por tanto la prueba beta es una aplicación "en vivo" del software en un ambiente que no puede controlar el desarrollador. El cliente registra todos los problemas (reales o imaginarios) que se encuentran durante la prueba beta y los reporta al desarrollador periódicamente. Como resultado de los problemas reportados durante las pruebas beta, es posible hacer modificaciones y luego preparar la liberación del producto de software a toda la base de clientes.

- Prueba de sistemas: Se buscan discrepancias entre el sistema y sus objetivos originales, a partir del análisis de objetivos y en base a la documentación destinada al usuario.
  - Pruebas de seguridad: La prueba de seguridad consiste en tratar de formular casos de prueba para burlar los controles del sistema de seguridad. Debe probarse al menos con las configuraciones mínima y máxima posibles. La penetración abarca un amplio rango de actividades: hackers, empleados resentidos, individuos deshonestos.
  - Pruebas de alcance: Determinar si cada enunciado mencionado en los objetivos, ha sido implementado realmente.
  - Pruebas de volumen: Se busca mostrar que el programa no puede manejar los volúmenes de datos especificados en los objetivos.
  - Pruebas de esfuerzo: Someter al programa a grandes cargas o a esfuerzos considerables en un periodo de tiempo corto.
  - Pruebas de facilidad de uso: Ver si el programa es amigable y fácil de usar.
  - Pruebas de comportamiento: Probar la eficiencia del programa.
  - Pruebas de almacenamiento: Tratar de mostrar que el programa no cumple los objetivos de almacenamiento de memoria principal y secundaria y tamaño de archivos temporales.
  - Prueba de compatibilidad/conversión: Se quiere demostrar que los objetivos de compatibilidad no han sido logrados y que los procedimientos de conversión fallan.
  - Prueba de instalación: Su propósito no es encontrar errores de programación sino errores de instalación. Comprobar que el programa luego de instalado, se abre bien, se cargan archivos y bibliotecas, se configura correctamente el equipo, etc.
  - Pruebas de recuperación: La recuperación es una prueba del sistema que fuerza al software a fallar en varias Formas y que verifica que la recuperación se realice de manera adecuada.
  - Prueba de capacidad de servicio.
  - Prueba de configuración: con frecuencia, el número de configuraciones posibles es demasiado grande para intentar una prueba de cada una de ellas, pero el programa debe probarse al menos con cada tipo de dispositivo y con las configuraciones mínima y máxima posibles.
  - Prueba de documentación.
  - Prueba de procedimiento: Se deben probar cualquier procedimiento humano.

  Realización de las pruebas de sistemas: Se recomienda que las pruebas la realicen una compañía distinta. Las pruebas no deben ser efectuadas por la propia organización.

Planeamiento y control de pruebas:

Es una parte decisiva de la organización del proceso de prueba. Los principales componentes de un buen plan son:

1. Objetivos: Deben definirse los objetivos de cada fase de la prueba.
2. Criterios de terminación: Deben especificarse los criterios para determinar cuándo se puede considerar terminada cada fase de la prueba.
3. Cronograma: Se necesita fijar los tiempos necesarios para cada fase. Este cronograma indicará cuando se diseñarán, escribirán y ejecutarán los casos de prueba.
4. Responsabilidades: Para cada fase será establecido quién diseñará, escribirá, ejecutará y verificará los casos de prueba, y quienes han de corregir los errores que fueran encontrados.
5. Configuración del equipo: Si se necesitan configuraciones especiales de equipo o se requiere algún periférico en particular, será necesaria una planificación que describa los requerimientos, cómo se cumplirán y cuándo se utilizarán.

Criterios para la terminación de las pruebas:

La condición de terminación para la prueba de función podría estar dada por las condiciones siguientes:

Los casos de prueba se obtienen de (1) análisis de valores límite y (2) conjetura de errores, y todos los casos de prueba resultantes son infructuosos en la detección de errores.

Esta primera categoría de criterios fija métodos de diseño de casos de prueba, en lugar de proporcionar una meta y de permitir la elección del camino más apropiado para lograrla.

La segunda categoría de criterios, y quizá la más valiosa, es establecer los requerimientos de terminación en términos positivos. El objeto que se busca con las pruebas es encontrar errores; este tipo de criterio refuerza la definición de prueba. Tiene, sin embargo, dos problemas. Un problema es como determinar el número de errores a ser detectados.

La obtención de este número requiere:

- Una estimación del número total de errores del programa.
- Una estimación del porcentaje de estos errores que pueden encontrarse fácilmente mediante las pruebas.
- Estimación de que fracción de errores se originan en procesos particulares de diseño y durante cuales fases de las pruebas se puede esperar que sean detectados estos errores.

El tercer tipo de criterio consiste en representar gráficamente el número de errores encontrados por unidad de tiempo durante la fase de pruebas. Examinando la forma de la curva se puede a menudo determinar si conviene continuar una fase de prueba o terminarla y comenzar la fase siguiente.

El mejor criterio es una combinación de los tres tipos mencionados más arriba.

Para la prueba de componentes, el mejor criterio es probablemente el primero. En las fases de pruebas de función y de sistema, la regla para establecer la terminación de la prueba podría ser parar cuando se detecte un número predeterminado de errores, o cuando haya trascurrido el tiempo fijado para ello, eligiendo la situación que ocurra más tarde, siempre que un análisis del gráfico del número de errores detectados por unidad de tiempo indique que la prueba se ha tornado improductiva.

Debugging

Es la actividad que se desarrolla después de ejecutar un caso de prueba exitoso. Consta de dos partes:

- Determinar la naturaleza exacta y la ubicación del presunto error dentro del programa: Ubicar el error consta del 95% del problema, ya que es una tarea difícil.
- Corregir el error encontrado.

Se considera al debugging como la tarea más abrumadora (el programador está presionado por la organización y el cliente para corregir el error, y auto-presionado para lograr corregirlo rápidamente; además la ubicación de un error puede ser cualquier sentencia del programa) y psicológicamente difícil (corregir un error es un indicio de imperfección laboral).

Métodos de debugging:

- **Sistemáticos:**
    - Por deducción: El proceso de deducción consiste en partir desde ciertas premisas o teorías generales, utilizando los procesos de eliminación y clasificación para llegar a una conclusión (la ubicación del error). Los pasos son:
        - Enumerar las causas e hipótesis posibles: desarrollar una lista de todas las causas concebibles del error.
        - Usar los datos para eliminar causas posibles: analizando los datos y buscando contradicciones se intenta eliminar todas las posibles causas menos una.
        - Clarificar las hipótesis restantes: usar las pistas disponibles para transformar la teoría en algo más específico.
        - Probar las hipótesis restantes: la hipótesis se prueba mediante el estudio de los datos y pistas originales, controlando que ésta explique completamente la existencia de lo que se observe.
    - Por inducción: Es la suposición de que en muchos casos la mayor parte de los errores pueden ubicarse mediante un razonamiento cuidadoso, sin necesidad de utilizar la computadora. Este proceso es el de inducción, en el cual se avanza desde lo particular hacia lo general. Es decir, se comienza con las pistas (los síntomas del error) y buscando sus relaciones se puede, a menudo, llegar a encontrar el error. Se siguen los siguientes pasos:
        - Ubicación de los datos pertinentes: es la enumeración de todo lo que se sabe que el programa hace correctamente y también de lo que se sabe que hace incorrectamente.
        - Organización de los datos: es la presentación de la información para permitir la observación de estructuras.
        - Formulación de una hipótesis: estudiar las relaciones entre los síntomas y formular una o más hipótesis sobre la causa del error.
        - Prueba de la hipótesis: la hipótesis se prueba mediante el estudio de los datos y pistas originales, controlando que ésta explique completamente la existencia de lo que se observe.
    - Por rastreo hacia atrás: Un método efectivo para encontrar errores en programas pequeños es rastrear hacia atrás a través de la lógica del programa hasta que se descubre el punto en que la lógica se desvía de lo correcto.

- Por medio de pruebas: Se pueden considerar dos tipos de casos de prueba: casos de prueba para probar en los cuales el propósito es exponer errores hasta el momento no detectados y casos de prueba para debugging donde el propósito es obtener información útil en la ubicación de un determinado error. La diferencia entre ambos es que los primeros tienden a ser casos "gordos" (se trata de cubrir tantas condiciones como sea posible con un número reducido de casos) y los segundos tienen la tendencia a ser "delgados" (se trata en lo posible de cubrir una condición única o unas pocas condiciones con cada caso).
  Después de descubrir un síntoma de un error, se emplea este método para escribir variantes del caso de prueba original, tratando de ubicar con exactitud el error.

- **Asistemáticos:** Sólo se recomienda el uso de métodos asistemáticos cuando fallan otros métodos o como suplemento (no substituto) de los procesos reflexivos. Son métodos poco eficientes pero muy usados, quizá porque requieren de menor esfuerzo mental.

Se pueden dividir en tres categorías:

- Vuelco de memoria: Analizar un vuelco de memoria (usualmente una presentación llana de todas las direcciones de almacenamiento en formato hexadecimal) es probablemente el más ineficiente. Los problemas son:
  - Dificultad al establecer correspondencia entre las direcciones de almacenamiento y las variables en el programa fuente.
  - La mayor cantidad de la información es irrelevante.
  - Un vuelco constituye una imagen *estática* del programa, sin embargo se necesita estudiar la *dinámica* del programa.
  - El vuelco puede enmascarar la información que se está buscando.
  - Falta de métodos para encontrar la causa de un error.
- Esparcimiento de sentencias: Consiste en esparcir sentencias dentro del programa (introducir cambios) para producir mensajes o mostrar el valor de ciertas variables. Muestra la *dinámica* del programa y permite examinar la información relacionada con el programa fuente pero sin embargo presenta defectos:
  - Método de prueba y error que no intenta pensar en el problema que se trata de resolver.
  - Puede producir una enorme cantidad de datos para analizar.
  - Requiere la introducción de cambios en el programa lo que puede enmascarar el error.
  - Su empleo en programas extensos puede ser de un enorme costo y en determinados programas imposible.
- Debuggers: En lugar de introducir cambios en el programa, se analiza la *dinámica* de éste usando herramientas de debugging del lenguaje de programación empleado. Puede agregar "puntos de detención" que hacen que se suspenda la ejecución del programa y permite al programador examinarlo en ese instante.

PRINCIPIOS DE "DEBUGGING"
Principios de búsqueda de errores

Pensar

El método más efectivo para hacerlo es el análisis de la información asociada con los síntomas del error. Para realizar el proceso en forma eficiente se debe ser capaz de ubicar la mayoría de los errores sin necesidad de acudir a la computadora.

Si se arriba a *un impasse, descansar*

Si no se puede resolver el problema en un tiempo razonable, conviene abandonarlo y trabajar en otra cosa, puesto que la eficiencia mental de todos modos ha disminuido. Después de "olvidarlo" por un tiempo puede ocurrir que el subconsciente haya resuelto el problema o la mente consciente esto suficientemente despejada como para encarar un nuevo examen de los síntomas.

Si se arriba a un impasse, describir el problema a otra persona

Es frecuente que la simple descripción del problema a un buen interlocutor haga que se encuentre la solución súbitamente, sin que sea necesaria la asistencia de aquel.

Usar herramientas de "debugging" *sólo como segundo recurso*

Usarlas como una ayuda y no como un substituto del raciocinio (pensamiento).

Evitar experimentos. Usarlos como último recurso

Este enfoque incierto ni siquiera puede ser considerado debugging, más bien representa un acto de esperanza ciega. Este método, a menudo complica el problema por el agregado de nuevos errores al programa.

Principios para reparación de errores

Donde existe una falla, es probable que haya otra

Los errores tienden a agruparse. Cuando se repara un error, conviene inspeccionar su vecindad observando toda otra circunstancia que pueda parecer sospechosa.

Arreglar el error, no solamente un síntoma de él

Si la corrección propuesta no satisface *todas* las pistas relacionadas con el error, se estará solo corrigiendo una parte del problema.

La probabilidad de que la corrección esté bien no es de 100 %

La codificación agregada a un programa para corregir un error no puede ser tomada como correcta. Consideradas sentencia por sentencia, las correcciones son mucho más propensas a contener errores que la codificación original. Otra consecuencia es que las enmiendas deben ser probadas, quizá, más cuidadosamente que el programa original.

La probabilidad de que la enmienda sea correcta disminuye con el tamaño del programa

En un programa grande, muy usado, uno de cada seis nuevos errores descubiertos ha sido cometido en una corrección previa hecha al programa.

Atención a la posibilidad de que la corrección de un error produzca un nuevo error

Una consecuencia es que no sólo la situación del error debe ser probada después de la corrección sino que debe realizarse una prueba de deterioro para determinar si se ha introducido alguno nuevo.

El proceso de reparación de los errores debe hacer que el proyecto vuelva temporariamente a la etapa de diseño.

Dada la propensión de las correcciones a contener errores, el sentido común dice que los procedimientos, metodologías y formalismos usados en el proceso de diseño, cualesquiera sean ellos, deberán ser también aplicados al proceso de corrección de errores.